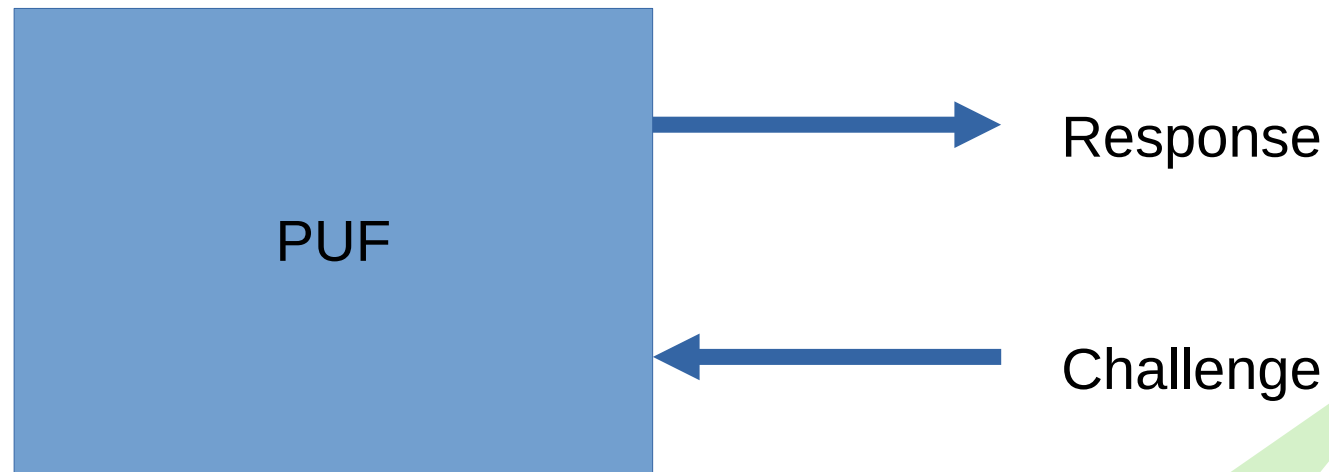


PUF The Magic Dragon
Smoked From his Nose
He slept on beach
And snored when he doze

David Johnston dj@primitivesecurity.com

- PUF : Physically Unclonable Function

- It's a function. $F(\text{input}) \rightarrow \text{Output}$
- It's unclonable – sounds like nonsense. It is, but..
- The process of building one yields a different PUF
- Without knowledge of the secret content of a PUF, you can't clone one

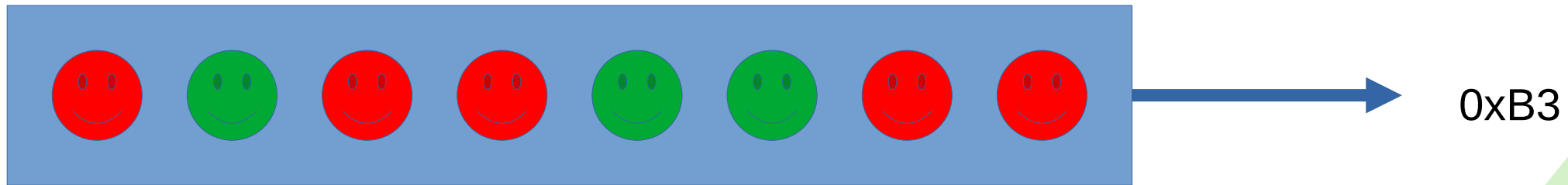


Whaddat?



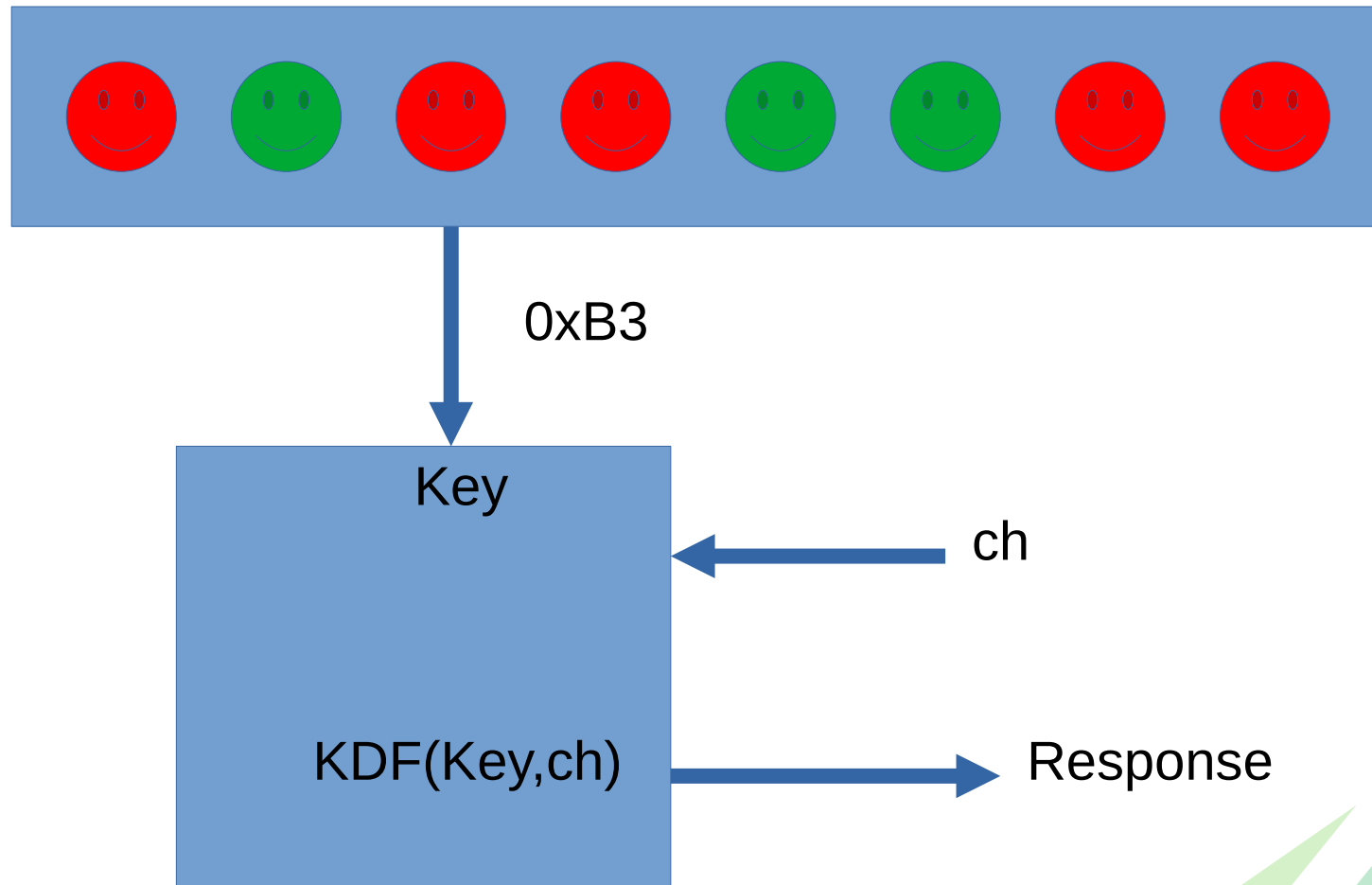
- It looks like a big memory of random numbers
- The challenge is the index
- The response is a random number from the memory
- Different instances see a different sequence
- Different challengers can see a difference sequence if designed to do that

What Does a PUF Look Like?

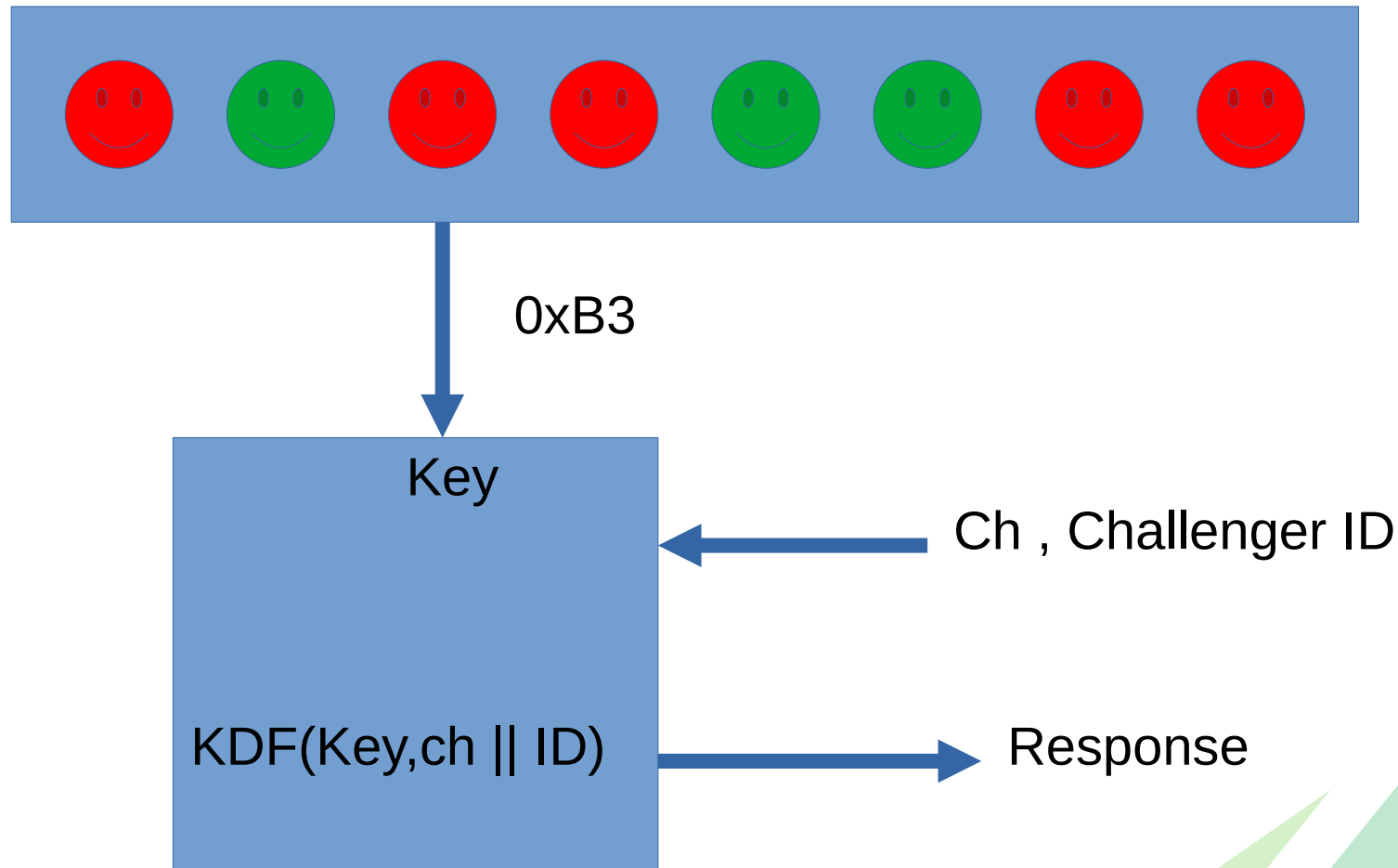


- Nail some tokens to a plank. Choose red or green randomly for each token.
- Red=1, Green=0
- This PUF reads 10110011 or 0xB3
- But you could clone that

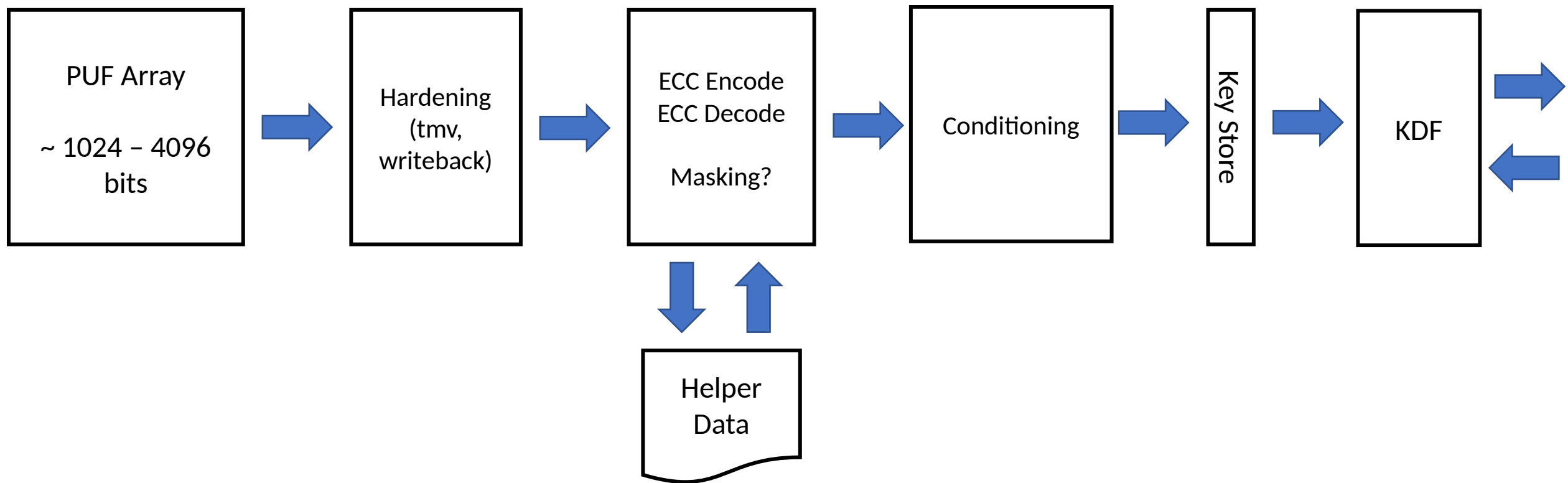
Lets make a PUF



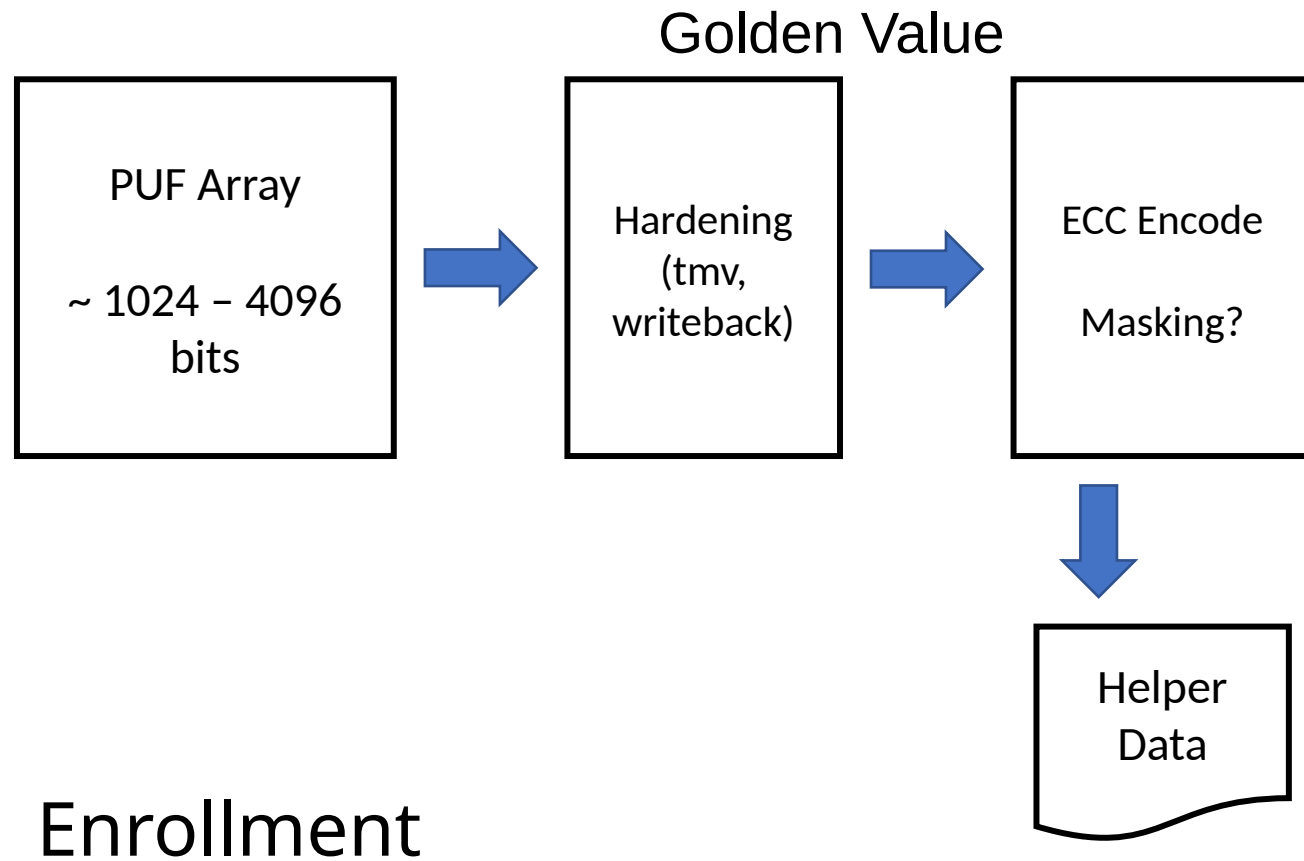
Now the challenger can't learn the PUF value



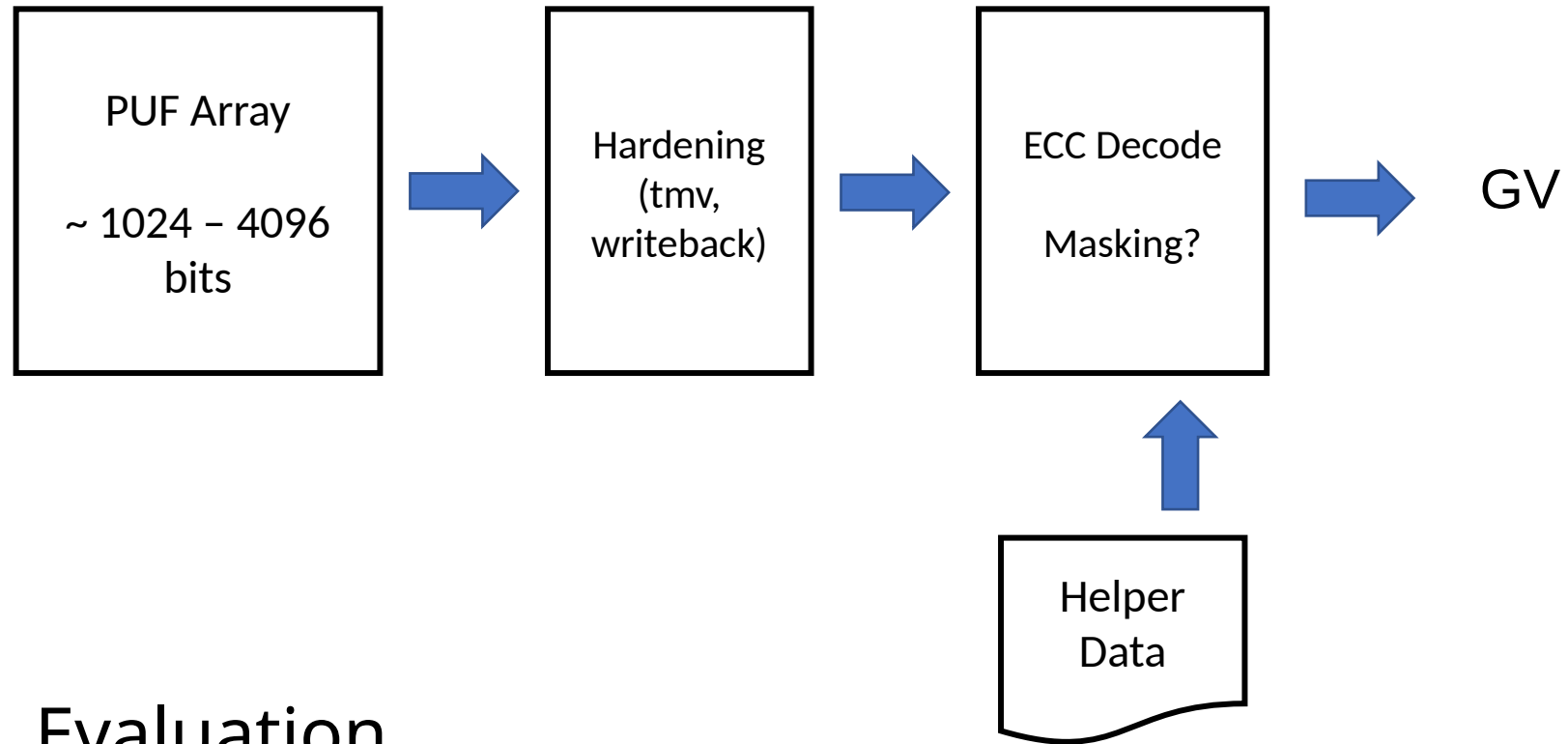
We want different challengers to see different PUFs



A More Realistic PUF



GV + Errors

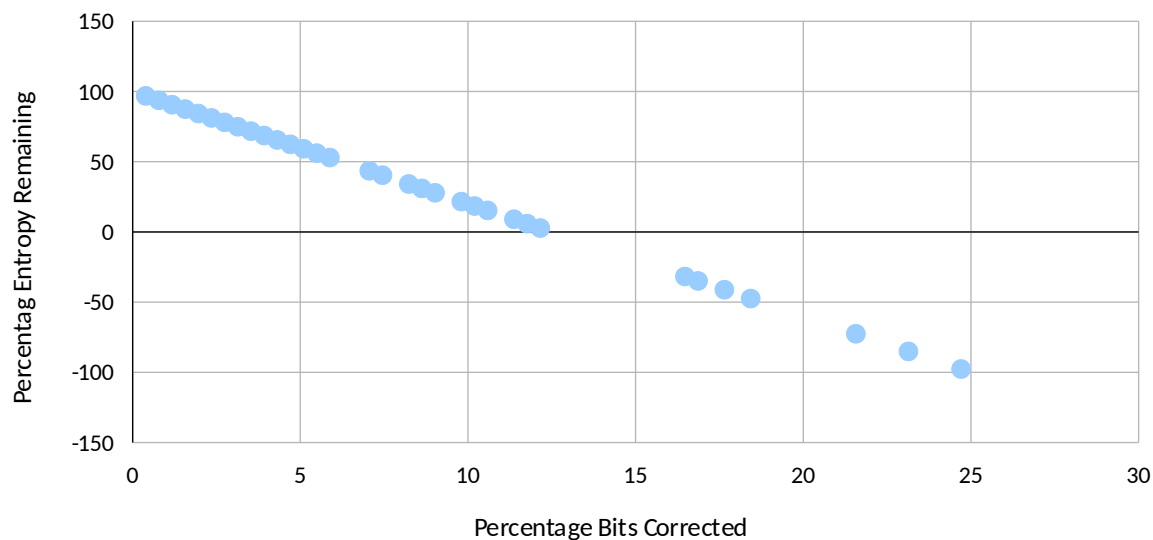


Evaluation

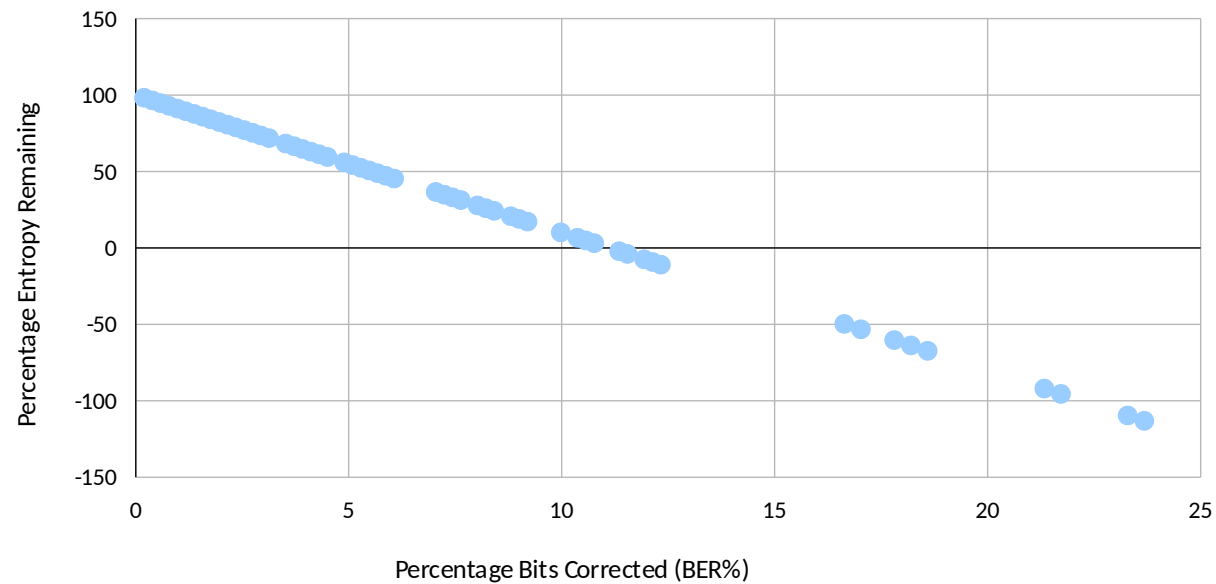
- Silicon changes as it ages
- But it changes a lot faster when current is passing through a transistor
- An SRAM cell with manufacturing bias such that it power onto the value 1, will while powered, weaken the stronger transistor that was driving it to 1, ultimately leading it to power on as a zero if the cell remains powered for long enough
- A HCI cell when powered on, degrades in the direction that it is in, strengthening the reliability of the PUF cell
- “Hardening” is the process of accelerated aging. In an SRAM cell the reverse of the golden state is written back to the SRAM before hardening with increase voltage and temperature while it’s powered. In a HCI cell, the cell is just left at its default state and hardened before the golden state is read out.
- SRAMs in PUFs will degrade with use. If the correct aging calculations have not been done, then SRAM PUFs may fail some time after deployment

Silicon Aging

For 255 bit blocks



For 511 bit blocks



BCH Entropy Loss

<https://github.com/dj-on-github/pufattack>

pufattack

There was once a paper from a major semiconductor manufacturer, describing a PUF. It could be seen that the entropy leaked through the chosen BCH scheme was greater than the size of the PUF

This code is a proof of concept to show a PUF made with that BCH scheme allows the PUF value to be retrieved from the check bits alone.

Why is this possible? The errors in a BCH used on a communication channel can occur in the data or the check bits. In a PUF the check bits are usually reliable (E.G. stored in fuses or flash memory) and the errors only occur in the PUF cells. So if the size of the the BCH table is big enough, each row in the BCH table uniquely identifies a PUF value.

This is what the code shows. It makes a random PUF value. Builds the check bits. Then uses only the check bits to compute the PUF value.

The vendor was told and they changed the algorithm.

All Commercially Available PUFs are insecure (except for mine)

```
# A bch table to look up the ecc
bchtable = [0 for _ in range(2**24)]

for i in range(2**24):
    if (i % 10000000)==0:
        print(i)

    byte1 = i & 0xff;
    byte2 = (i >> 8) & 0xff
    byte3 = (i >> 16) & 0xff

    bytes = bytearray([byte1, byte2, byte3])
    ecc = bch.encode(bytes)
    ecci = bytes_to_int(ecc)

    bchtable[i] = ecci
```

```
# Use the ECCs to predict the PUF
for ecc,ecci in zip(eccs,eccis):
    for tablepuf,tableecc in enumerate(bchtable):
        if (tableecc == ecci):
            print(" Found ECC %010x ==> PUF %06x" % (ecc,tablepuf))
```

```
# Make some PUFs
eccis = list()
eccs = list()
for i in range(10):
    puf = random.getrandbits(24)
    byte1 = puf & 0xff;
    byte2 = (puf >> 8) & 0xff
    byte3 = (puf >> 16) & 0xff

    pufbytes = bytearray([byte1,byte2,byte3])
    pufi = bytes_to_int(pufbytes)

    ecc = bch.encode(pufbytes)
    ecci = bytes_to_int(ecc)

    eccs.append(ecc)
    eccis.append(ecci)

    print("PUF %06x  ECC %010x" % (pufi,ecci))
```

All Commercially Available PUFs are insecure (except for mine)

The best SRAM cell is the worst PUF cell.

The best PUF cell is the worst SRAM cell.

The engineering goals are fundamentally opposed.

SRAM : Closer to 50/50 1/0 power up balance the better, $\sigma_m \ll \sigma_n$

PUF : The closer to a solid 100/0 or 0/100 1/0 the better. $\sigma_m \gg \sigma_n$

Exceeding 12.05% error correction capability and you have lost all your entropy

SRAM PUFs need at least 25% error correction. Typically more.

IID uses Golay codes. Never use IID

Read the array, derive the key, store the key safely, power down the array. Don't let users cause the array to be powered.

All Commercially Available PUFs are insecure (except for mine)

Engineer a lower error PUF cell : There is no hope in improved masking schemes
We developed Hot Carrier Injection PUF cells :

<https://ieeexplore.ieee.org/document/10118128>

Rachael J. Parker; Jyothi Bhaskarr A. Velamala; Kuan-Yueh James Shen; David Johnston;
Yao-Feng Chang; Stephen M. Ramey

PUFs are an exercise in advanced obfuscation. Cryptography is incidental.
OBFUSCATE! OBFUSCATE! OBFUSCATE!
Accept the engineering challenge of obfuscation. Kirchoff's law does not apply

Employ lower leakage helper data schemes : HCC. RM, RM-HCC

Rules for Good PUFs

- HCC Needs a HCI PUF array
- HCI cells used as PUF bits have lower BER than HCI cells used as fuse bits
- Write the helper data back to the array. It's HCI fuses after all.
- Inner code is low power to correct the low BER PUF bits
- Outer code only corrects the high BER inner fuse bits. It doesn't leak any more entropy.



The Half Concatenated Code

Algorithm 1: Key Generation using HCC

Input: $[n_1, k_1, d_1]$ code C_1 and $[n_2, k_2, d_2]$ code C_2 ; Set of universal *Hash* function $\mathcal{H}$, PUF response R

Output: (w_1, w_2) and a key K

- 1 Generate uniformly at random codeword v_1 from C_1
 - 2 Form the bitstrings P_1 and P_2
/* P_1 (k_1 bits) $\leftarrow$ message bits of v_1 and
 P_2 $((n_1 - k_1)$ bits) $\leftarrow$ parity bits in v_1 */
 - 3 $v_2 \leftarrow \text{Encode}_{C_2}(P_2)$
 - 4 $\ell_1 \leftarrow (P_1 \| v_2)$
 - 5 $w_1 = \ell_1 \oplus R$
 - 6 $K \leftarrow h_i(R)$
 - 7 $w_2 = i$
-

Algorithm 2: Key Reproduction using HCC

Input: $[n_1, k_1, d_1]$ code C_1 and $[n_2, k_2, d_2]$ code C_2 ; Set of universal *Hash* function $\mathcal{H}$, PUF response R ; Helper data W (w_1, w_2)

Output: Key K

/* Let ω be the binary representation of w_1 ;
 $\omega = (\omega_1, \omega_2, \dots, \omega_{p_1}, \omega_{p_1+1}, \dots, \omega_R)$ */

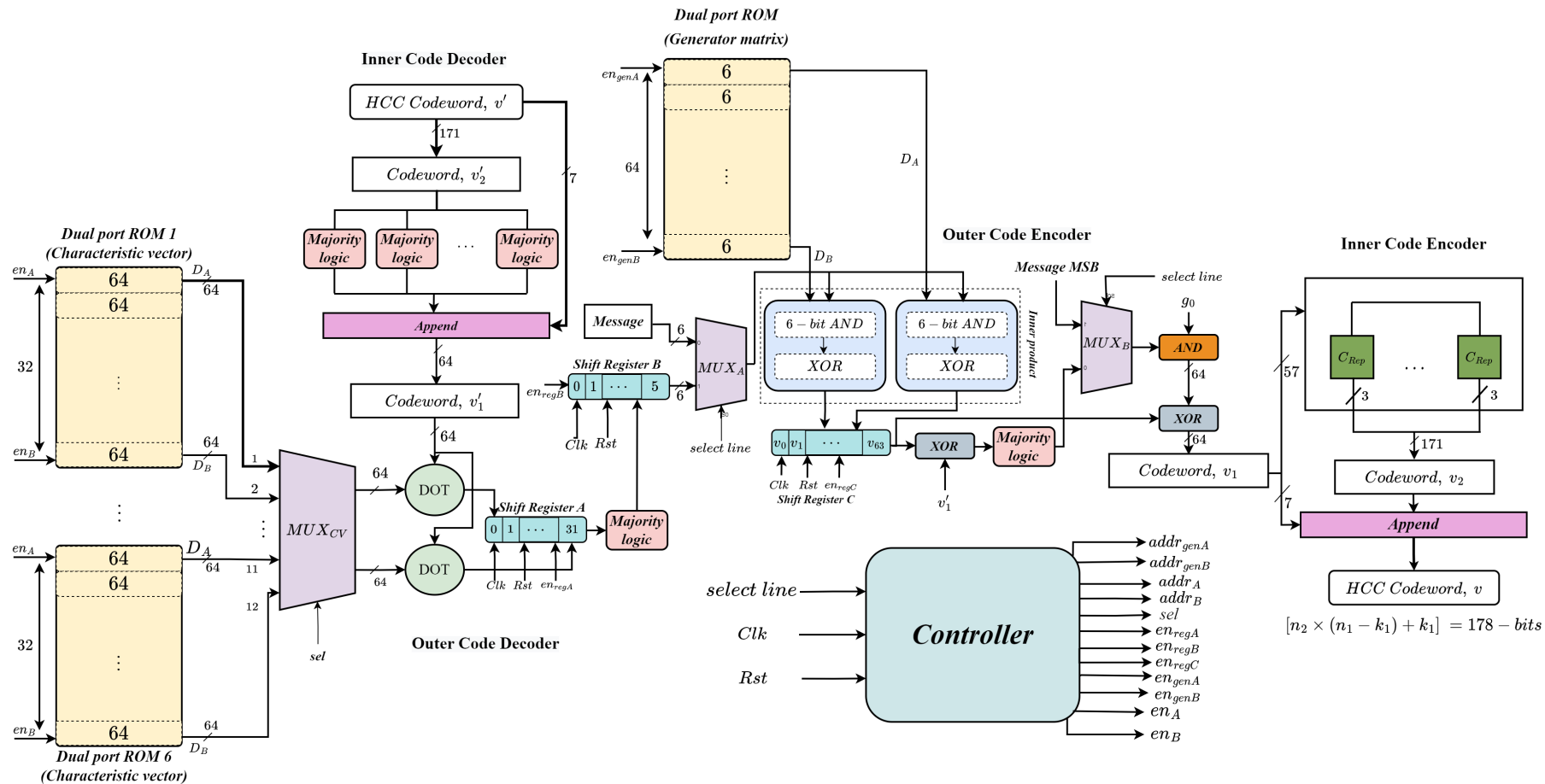
- 1 Let ω be partitioned into $\omega'_1 = (\omega_1, \omega_2, \dots, \omega_{p_1})$ and
 $\omega'_2 = (\omega_{p_1+1}, \dots, \omega_R)$
 - 2 $\omega_2 \leftarrow \omega'_2$ // Using Standard array to identify the error (coset leader)
 - 3 $m_2 \leftarrow \text{Decode}_{C_2}(\omega_2)$ // $m_2 \leftarrow$ parity-check bits
 - 4 $\ell'_2 \leftarrow (\omega'_1 \| m_2)$
 - 5 $\ell_2 \leftarrow \text{Decode}_{C_1}(\ell'_2)$ // Using Standard array to identify the error (coset leader); here, $\ell_2 \leftarrow$ is the recovered codeword
 - 6 $R \leftarrow w_1 \oplus \ell_2$ // Using w_1 and ℓ_2 , R can be recovered correctly
 - 7 $K \leftarrow h_i(R)$ // $i \leftarrow \omega_2$; $h_i \in \mathcal{H}$
-

Enroll and Evaluate. SKEDDEL Algorithms. That was easy.

SKEDEL for PUFs: Secure Key Extraction Diminishing Entropy Leakage from PUFs

A new hope!

Akashdeep Saha, Tishya Sarma Sarkar, Kuheli Pratihari, Siddhartha Chowdhury, Urbi Chatterjee,
Debdeep Mukhopadhyay, Vijayavardhan B, David Johnston



- Use PUFs for authentication and pairwise key establishment

PUF-RAKE: A PUF-Based Robust and Lightweight Authentication and Key Establishment Protocol DOI: [10.1109/TDSC.2021.3059454](https://doi.org/10.1109/TDSC.2021.3059454)

Or this: <https://arxiv.org/pdf/2311.04078>

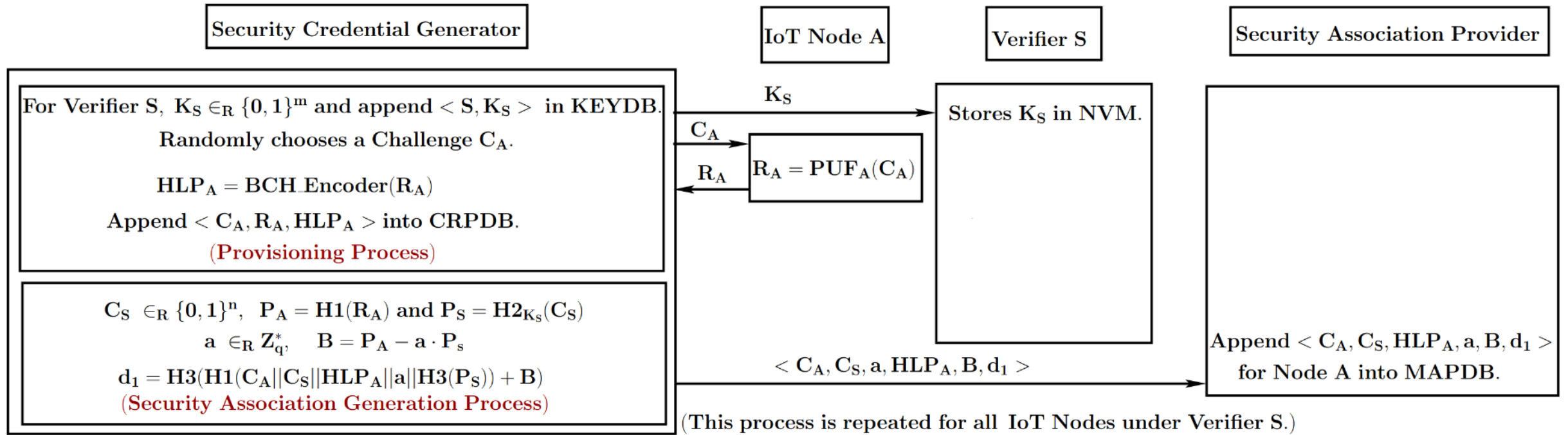
- These papers and their references show a disturbing bias towards IoT applications and toy PUFs. Ingrid (K.U. Leuven) cuts through the nonsense as usual: <https://eprint.iacr.org/2014/977.pdf>
- I'm reading the papers on PUF based KE to understand what can be done

For Secure Comms between two nodes, you want pairwise keys

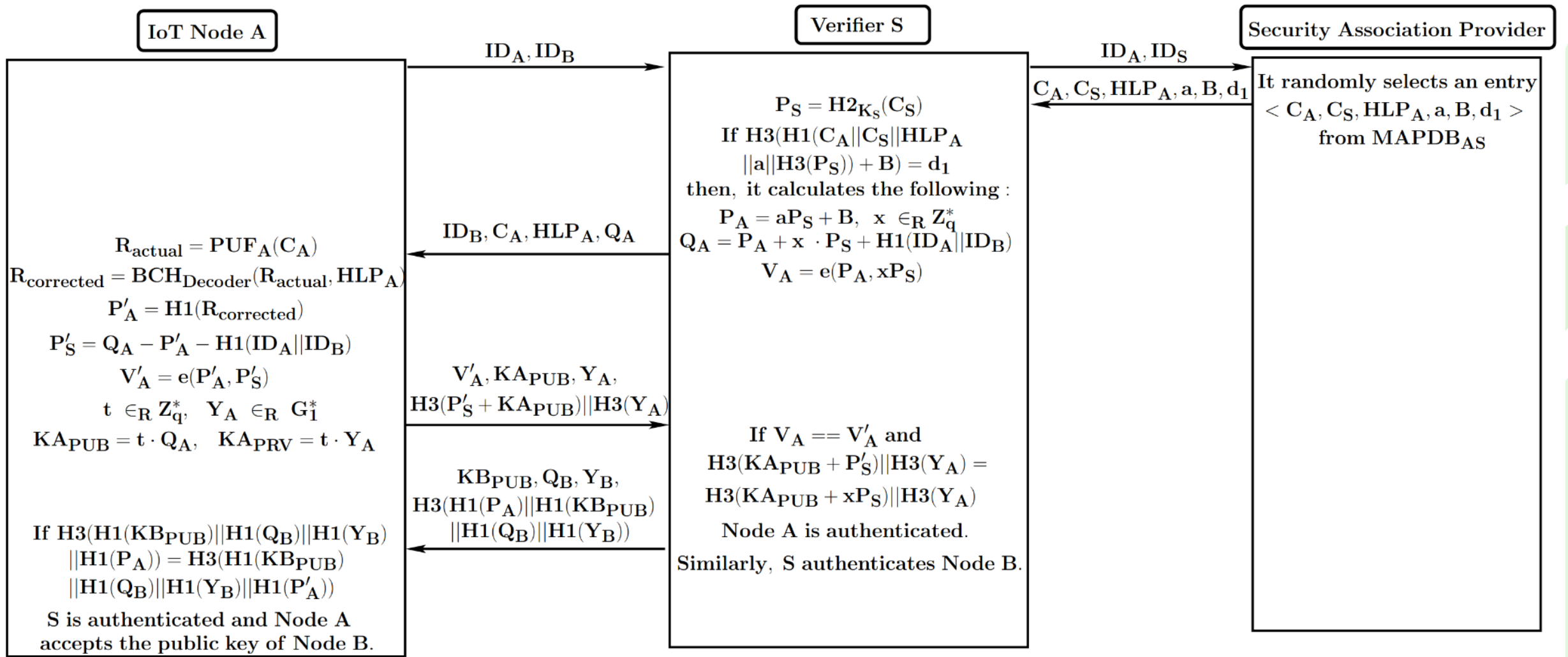
PUF+ IBE: Blending Physically Unclonable Functions with Identity Based Encryption for Authentication and Key Exchange in IoTs

Urbi Chatterjee, Vidya Govindan, Rajat Sadhukhan, Debdeep Mukhopadhyay,
Rajat Subhra Chakraborty, *Senior Member, IEEE*, Debashis Mahata, and Mukesh M Prabhu

- This one looks good. IBE with PUFs. The protocol needs untangling from the error correction but aside from that it looks ok if IBE is an acceptable model
- <https://eprint.iacr.org/2017/422.pdf>



- Enrollment
- HLP_A or $\text{BCH_Encoder}(R_A)$ isn't needed



- Auth and Key Exchange
- But $R_{\text{corrected}} = R_{\text{actual}}$ if a reliable PUF is used

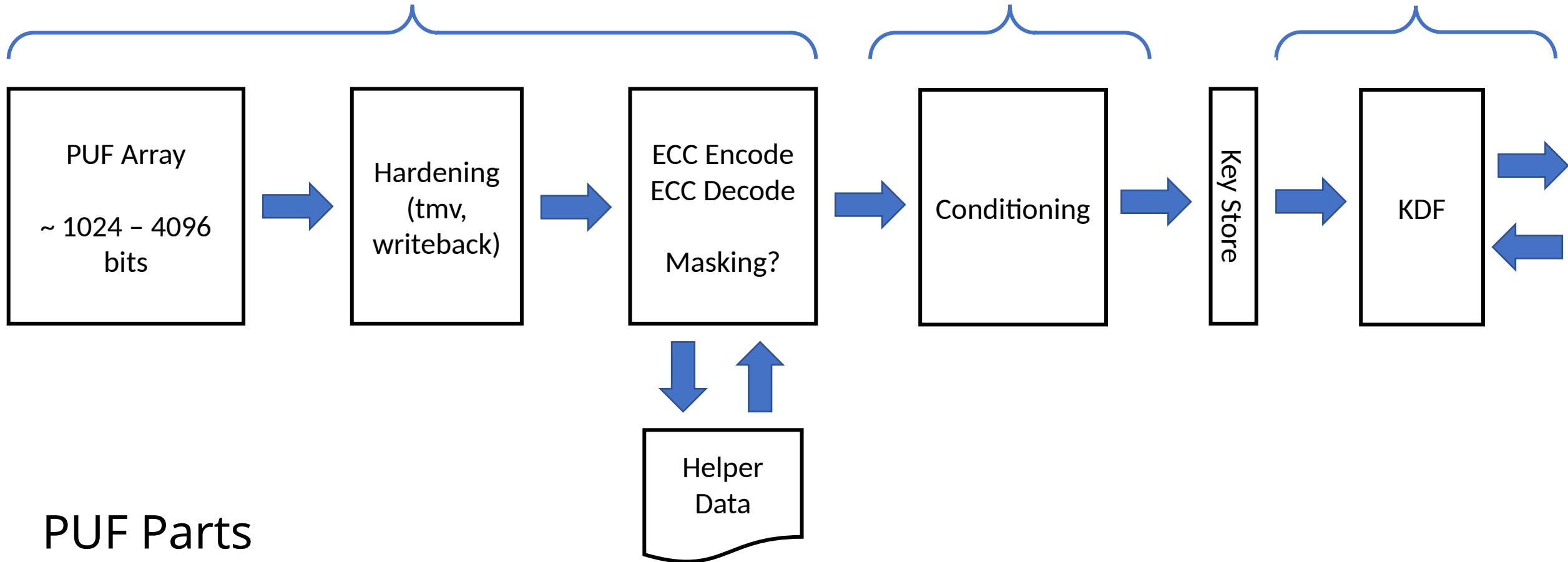
PUFs and SP800-90B

The Other Kind of RNG

Dynamic Random Numbers $\rightarrow$ RNG $\leftarrow$ SP800-90A,B,C / AIS20.31

Static Single Random Number $\rightarrow$ PUF $\leftarrow$???

This has no standard
But this is where you can really mess up
<https://github.com/dj-on-github/pufattack>



PUF Parts

SP800-90B:

Randomness Testing

A sequential dataset of at least 1 000 000 sample values obtained directly from the noise source (i.e., raw data) **shall** be collected for validation ². If the generation of 1 000 000 consecutive samples is not possible, the concatenation of several smaller sets of consecutive samples (generated using the same noise source) is allowed. Smaller sets **shall** contain at least 1000 samples. The concatenated dataset **shall** contain at least 1 000 000 samples.

You don't have 1,000,000 samples

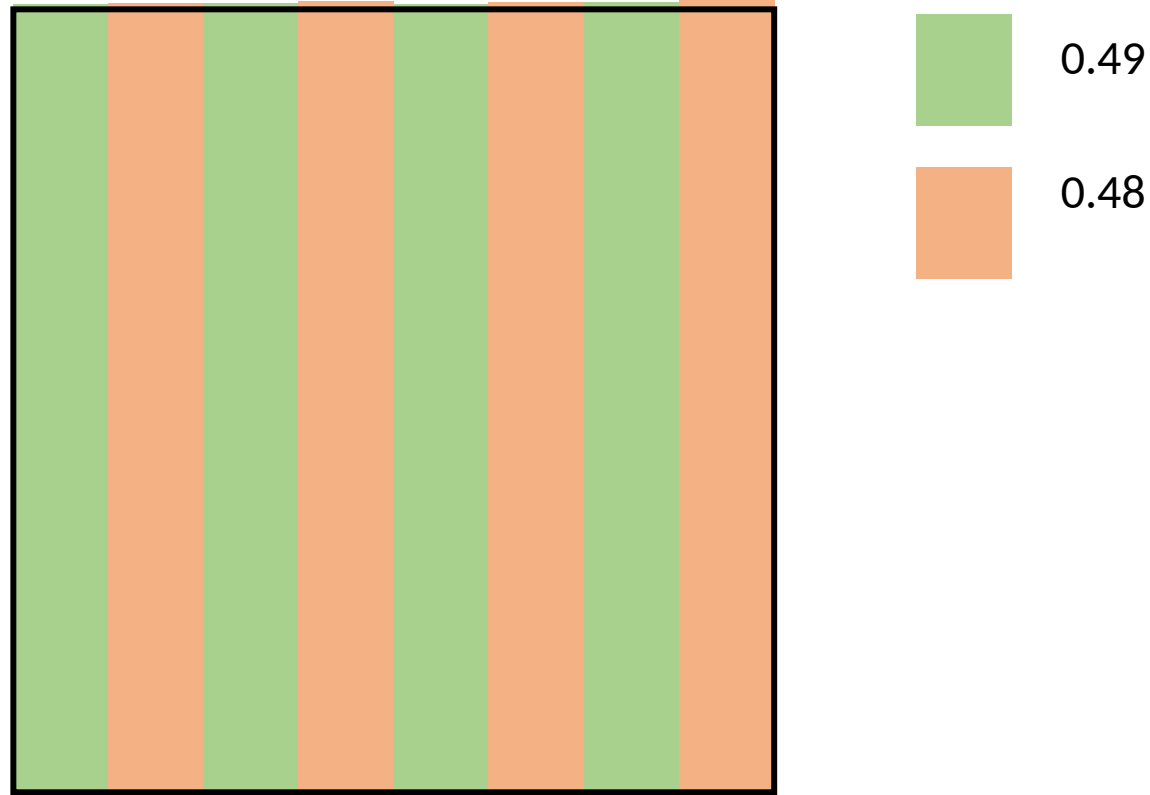
If you collect it and take it offline, your PUF is useless

If you collect from test chip PUFs, does that count?

If the test data is from 1,000 test chips, does that count?

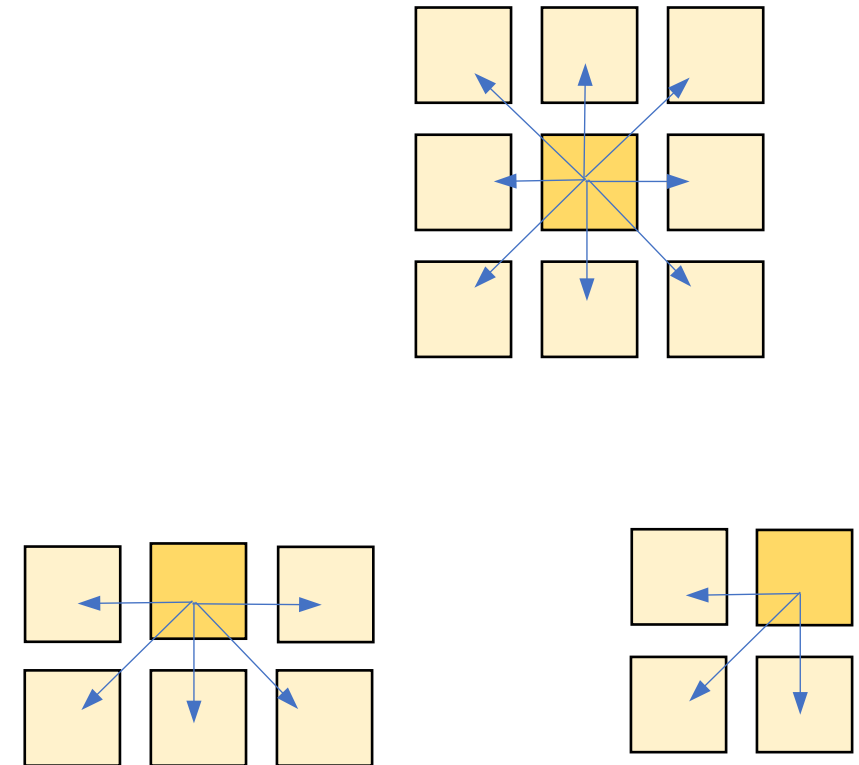
Actual Randomness Testing

PUF cell bias interacts with the silicon matrix in interesting ways:



- Intra and Inter Hamming distance over test chips
- Intra distance variation across enroll-eval temp delta
- Error counts from BCH give stability margin
- Public ICV data gives uniqueness verification : But hash the key, not the golden value

Predictor Tests from Spatial Correlation



Special treatment for edge and corner bits

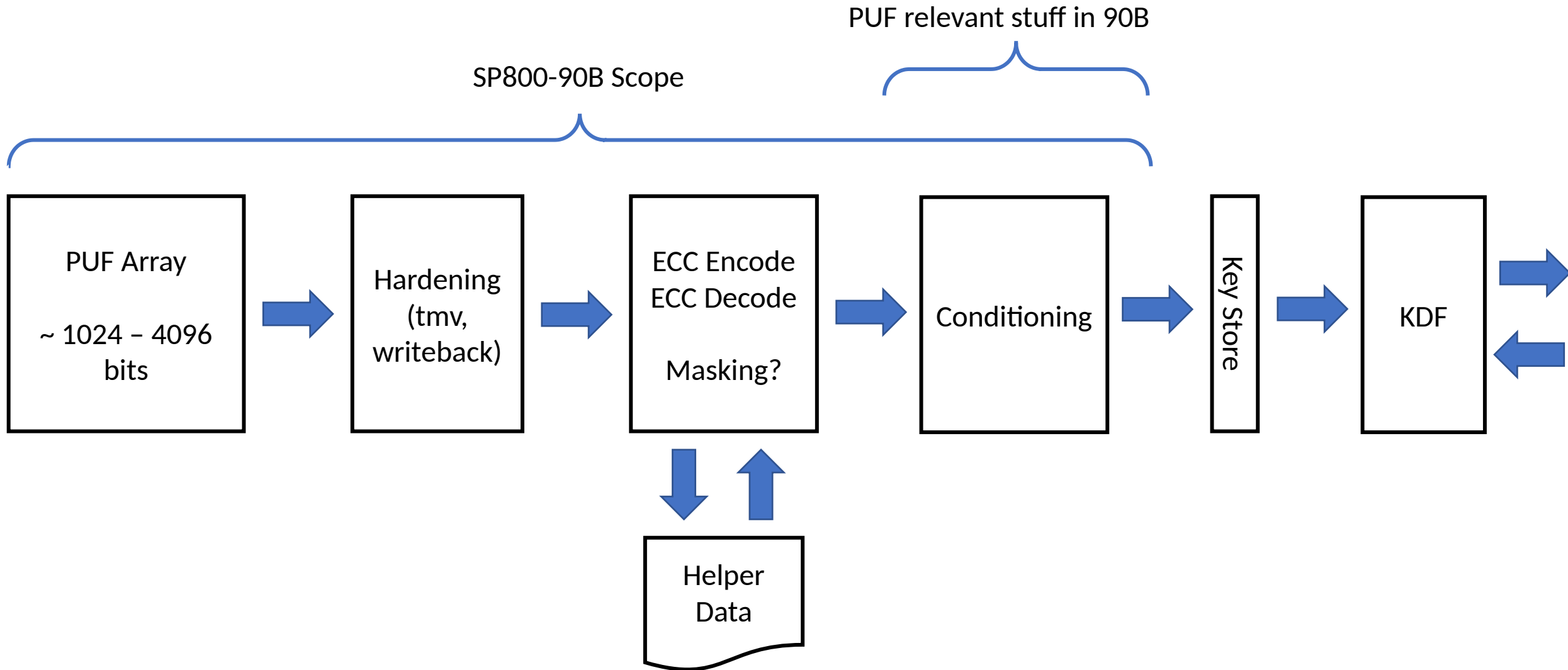
Restart Testing

SP800-90B:

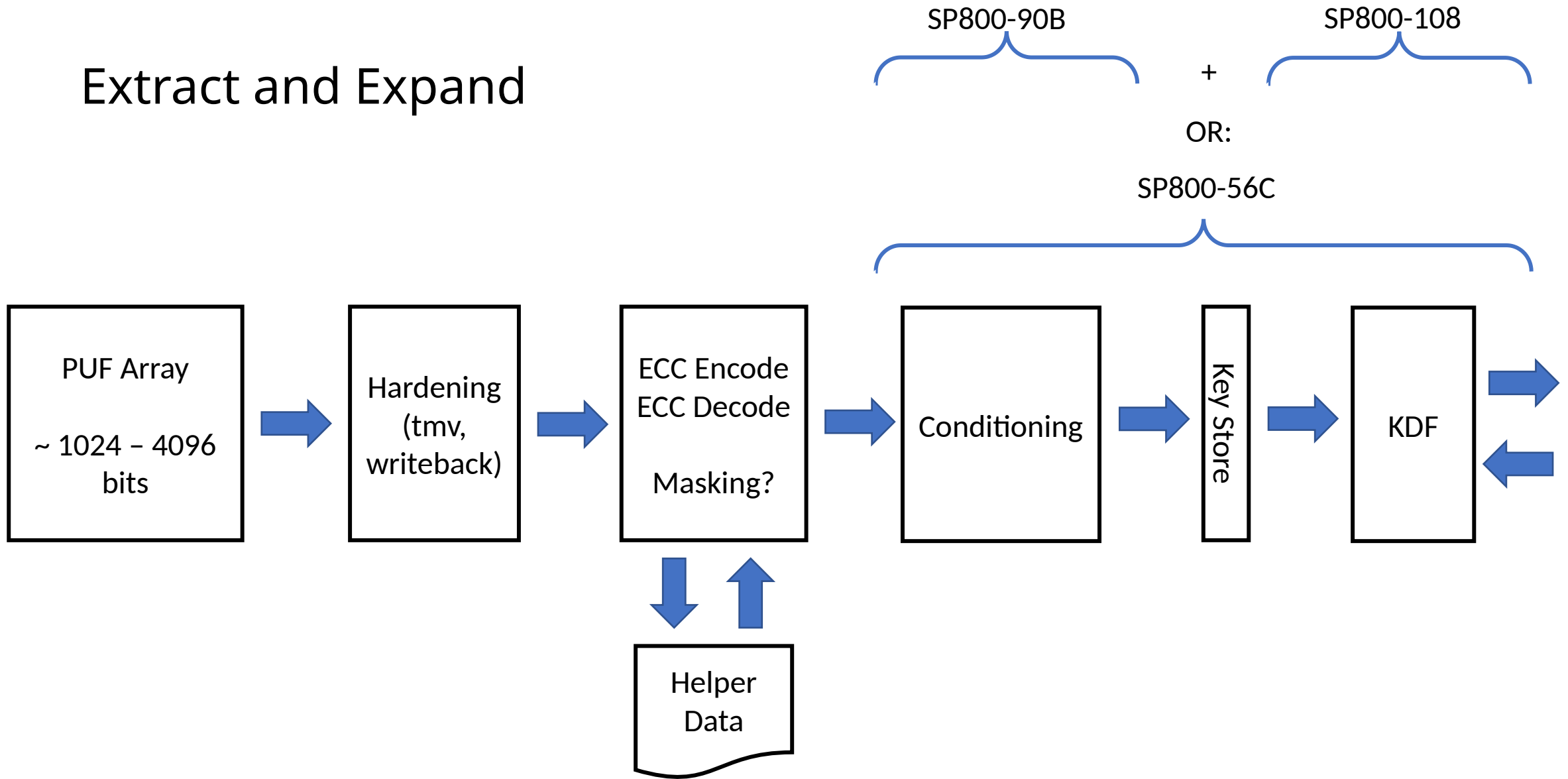
For the restart tests (see Section 3.1.4), the entropy source must be restarted 1000 times; for each restart, 1000 consecutive samples **shall** be collected directly from the noise source. The restart data **shall** be extracted whenever the noise source is ready and able to provide data that can be used for producing entropy source output. This data is stored in a 1000×1000 restart matrix M , where $M[i][j]$ represents the j^{th} sample from the i^{th} restart.

- A PUF will return the same data every time
- This is automatically fail the restart testing

90B Mismatch



Extract and Expand

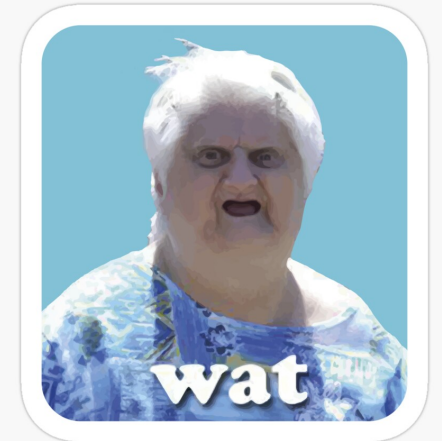


A Trap in 56C

Table 5: $\text{MAC}(\text{salt}, Z, \dots) = \text{AES-}N\text{-CMAC}(\text{salt}, Z)$ (For Randomness Extraction)

AES Variant used by AES-CMAC	Bit Length of salt for AES-CMAC	Bit Length of Extracted K_{DK}	Security Strength s supported (in bits)
AES-128	128	128	$112 \leq s \leq 128$
AES-192	192		
AES-256	256		

Note: Regardless of which AES variant is used by the AES-CMAC algorithm during randomness-extraction, the 128-bit AES block size determines the bit length of the resulting K_{DK} and the maximum security strength that can be supported by the use of the derived keying material (i.e., no more than 128 bits). To accommodate the use of this 128-bit K_{DK} as a key-derivation key, the CMAC mode of AES-128 **shall** be the PRF employed by the auxiliary PRF-based KDF in the subsequent key-expansion step.



- Top Tip:
 - You can extract multiple times to get bigger keys
 - Maybe use 90B + 108 to avoid this, or lean on your preferred NIST rep to fix this

Summary

- Trying to build a NISTy FIPSy PUF presents challenges
- SP800-90B is not useful for evaluating the entropy
- Algorithm choices can be made in isolation, but the assemblage of extractor + key derivation doesn't work with 56C
- What you build today might not match a future NIST spec



PRIMITIVE
SECURITY

David Johnston, dj@primitivesecurity.com, Hillsboro, OR, USA
+1 503 261 3658